

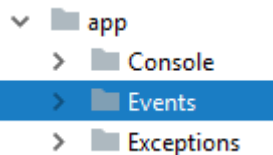
TP Events / Listeners

On a un sujet et des observateurs. Les observateurs doivent s'inscrire (on dit aussi s'abonner) auprès du sujet. Lorsque le sujet change d'état il notifie tous ses abonnés. Un observateur peut aussi se désinscrire, il ne recevra alors plus les notifications.

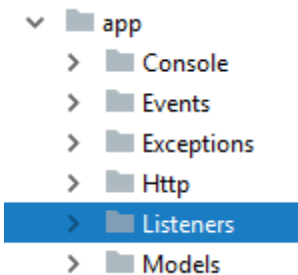
Organisation du code

Laravel implémente ce design pattern et le rend très simple d'utilisation. On va avoir :

- des classes **Event** placées dans le dossier **app/Events** :



- des classes **Listener** placées dans le dossier **app/Listeners** :



Ces dossiers n'existent pas dans l'installation de base, ils sont ajoutés dès qu'on crée la première classe avec Artisan qui dispose de commandes pour le faire :

<code>make:event</code>	Create a new event class
<code>make:exception</code>	Create a new custom exception class
<code>make:factory</code>	Create a new model factory
<code>make:job</code>	Create a new job class
<code>make:listener</code>	Create a new event listener class

Les événements déjà présents dans Laravel

Il n'y a pas que les événements que vous allez créer. Vous pouvez utiliser les événements existants déjà dans le framework qui en propose au niveau de l'authentification :

- Registered
- CurrentDeviceLogout
- OtherDeviceLogout
- Attempting
- Authenticated
- Login

- Failed
- Logout
- Lockout
- PasswordReset
- Validated
- Verified
- ...

On trouve aussi des événements avec Eloquent :

- retrieved
- creating
- created
- updating
- updated
- saving
- saved
- deleting
- deleted
- restoring
- restored
- ...

Pour tous ces cas vous n'avez donc pas à vous inquiéter du déclencheur qui existe déjà.

Une application de test

Pour nos essais créez une nouvelle application Events :

```
composer create-project laravel/laravel 017-Events
```

Créez une base, renseignez correctement le fichier **.env**. Installez Breeze et lancez les migrations :

```
composer require laravel/breeze --dev
php artisan breeze:install
```

Créez un utilisateur à partir du formulaire d'enregistrement :

The image shows a registration form with the following fields and content:

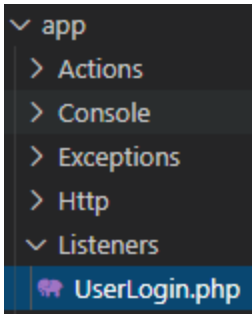
- Name:** Boubaker
- Email:** boubakeraddy@gmail.com
- Password:** (masked with dots)
- Confirm Password:** (masked with dots)
- Links:** [Already registered?](#) and a **REGISTER** button.

On crée un écouteur

On va utiliser l'événement intégré à Laravel qui se déclenche lorsqu'un utilisateur se connecte. La classe correspondante est **Illuminate\Auth\Events>Login**. Puisque l'événement existe on n'a pas à créer une classe **Event**. Par contre il nous faut une écoute (**Listener**) :

```
php artisan make:listener UserLogin
```

La classe **UserLogin** se crée ainsi que le dossier :



Voici le code généré :

```
<?php

namespace App\Listeners;

use Illuminate\Auth\Events>Login;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Queue\InteractsWithQueue;

class UserLogin
{
    /**
     * Create the event listener.
     */
    public function __construct()
    {
    }

    /**
     * Handle the event.
     */
    public function handle(Login $event): void
    {
        dd($event);
    }
}
```

Changez ainsi le code :

```
public function handle(Login $event): void
{
    dd($event->user->name . ' est connecté');
}
```

Cette fois à la connexion on obtient quelque chose dans ce genre :

```
"Boubaker s'est connecté." // app\Listeners\UserLogin.php:28
```

On peut donc ici effectuer tous les traitements qu'on veut, comme comptabiliser les connexions.

On crée un événement

Maintenant on va créer un événement et aussi son écouteur. On va imaginer qu'on veut savoir quand quelqu'un affiche la page d'accueil et mémoriser son IP ainsi que le moment du chargement de la page, en gros un peu de statistique.

On va commencer par créer une migration :

```
php artisan make:migration create_visits_table --create=visits
```

Avec ces créations de colonnes :

```
public function up(): void
{
    Schema::create('visits', function (Blueprint $table) {
        $table->id();
        $table->string('ip', 45);
        $table->dateTime('created_at');
    });
}
```

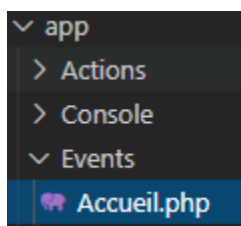
Et on lance la migration :

```
php artisan migrate
```

On crée l'événement :

```
php artisan make:event AccueilEvent
```

On le trouve ici avec création du dossier :



Avec ce code par défaut :

```
<?php

namespace App\Events;

use Illuminate\Broadcasting\Channel;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Broadcasting\PresenceChannel;
use Illuminate\Broadcasting\PrivateChannel;
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;

class AccueilEvent
{
    use Dispatchable, InteractsWithSockets, SerializesModels;

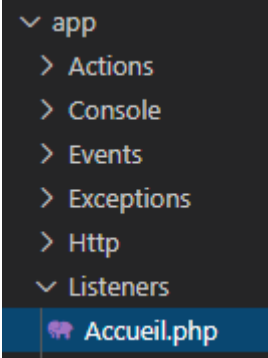
    /**
     * Create a new event instance.
     */
    public function __construct()
    {
        //
    }

    /**
     * Get the channels the event should broadcast on.
     *
     * @return array<int, \Illuminate\Broadcasting\Channel>
     */
    public function broadcastOn(): array
    {
        return [
            new PrivateChannel('channel-name'),
        ];
    }
}
```

On ne va pas toucher à ce code.

On crée aussi l'écouteur :

```
php artisan make:listener AccueilListener
```



app

- > Actions
- > Console
- > Events
- > Exceptions
- > Http
- ▼ Listeners
 - Accueil.php

On modifie ainsi le code :

```
<?php

namespace App\Listeners;

use App\Events\AccueilEvent;
use Illuminate\Support\Facades\DB;

class AccueilListener
{
    /**
     * Create the event listener.
     */
    public function __construct()
    {
        //
    }

    /**
     * Handle the event.
     */
    public function handle(AccueilEvent $event): void
    {
        DB::table('visits')->insert([
            'ip' => request()->ip(),
            'created_at' => now(),
        ]);
    }
}
```

J'utilise directement le Query Builder sans passer par Eloquent.

Il ne reste plus qu'à déclencher cet événement dans la route :

```
Route::get('/', function () {
    event(new AccueilEvent());
    return view('welcome');
});
```

Maintenant chaque fois que la page d'accueil est ouverte l'événement **Accueil** est déclenché et donc l'écoute **Accueil** en est informée. On sauvegarde alors dans la base l'IP et le moment du chargement :

id	ip	created_at
1	127.0.0.1	2022-02-22 19:55:31

Si le traitement de vos événements est long (envoi d'email, requête...) il est conseillé d'utiliser un système de file d'attente. Vous pouvez consulter la documentation correspondante.